

Semantic Similarity Engine

Project Specification — Phase 2, Week 7

Difficulty: Hard | Type: Practice Project (GitHub only, not on resume)

Dataset: MS MARCO v1.1 (HuggingFace Hub) | Execution: Google Colab T4 | Tracking: W&B;

Estimated Build Time: 3–4 hours

■ Problem Statement

Search engines powered by keyword matching (BM25, TF-IDF) fail silently on semantically equivalent but lexically different queries. A user searching *"heart disease prevention"* won't find a document titled *"cardiovascular risk reduction"* — even though the meaning is identical.

Semantic similarity engines solve this by representing both queries and passages as dense vectors in a shared embedding space, where **meaning determines proximity rather than word overlap**. This is the core retrieval component of every RAG system you will build in Phase 3.

You will build an end-to-end semantic search system that:

- Encodes a 10,000+ passage corpus using all-MiniLM-L6-v2 (Sentence Transformers)
- Indexes all embeddings in a FAISS index for fast nearest-neighbor retrieval
- Retrieves top-k passages for any natural language query
- Evaluates retrieval quality using MRR and Recall@k — the standard IR metrics
- Compares SBERT retrieval against a keyword baseline to quantify the improvement

Target: MRR@10 > 0.3, Recall@10 > 0.5 on 200 eval queries from MS MARCO.

■ **RAG Connection:** Every RAG system in Phase 3 (Weeks 9–13) uses this exact pipeline — encode query → FAISS search → retrieve passages → feed to LLM. Internalize it here before you build RAG from scratch.

■ Dataset Context

MS MARCO (Microsoft MACHine Reading COMprehension) is a large-scale information retrieval dataset built from Bing search queries and real web documents. It is the standard benchmark for passage retrieval tasks and is used in production search systems at Microsoft.

Property	Detail
Full dataset size	8.8M passages, ~1M queries
Subset used	15,000 training items → ~10,000 unique passages after dedup
Eval queries	200 queries with known relevant passage (is_selected flag)
Input	Natural language question (query) + candidate passages

Relevance signal	is_selected flag — which passage was used to answer the query
Source	load_dataset("ms_marco", "v1.1", split="train[:15000]")

Dataset Item Structure

```
{
  "query": "what is the capital of france",
  "passages": {
    "passage_text": ["Paris is ...", "France is ...", "The city of ..."],
    "is_selected": [1, 0, 0]  # which passage is the correct answer
  }
}
```

■ Why MS MARCO: Real queries from Bing users, real web passages, real relevance judgments. This is not a toy dataset — it is the benchmark used to evaluate BM25, DPR, ColBERT, and other production retrieval models.

■ Project Folder Structure

```
week7_semantic_similarity_engine/
■
■ ■ ■ notebooks/
■   ■ ■ ■ semantic_similarity_engine.ipynb  # single Colab notebook, all steps
■
■ ■ ■ index/
■   ■ ■ ■ corpus_index.faiss              # saved FAISS index
■   ■ ■ ■ passages.npy                   # saved passage strings
■
■ ■ ■ reports/
■   ■ ■ ■ figures/
■       ■ ■ ■ retrieval_metrics.png       # MRR@k and Recall@k bar chart
■       ■ ■ ■ similarity_heatmap.png     # cosine similarity heatmap (optional)
■
■ ■ ■ requirements.txt
■ ■ ■ README.md
```

■ The index/ directory is large — add *.faiss and *.npy to .gitignore. Document in README how to regenerate the index by running the notebook.

■ Technical Requirements

Phase 1 — Environment Setup & Data Loading

- Install in Colab: sentence-transformers, faiss-cpu, datasets
- Load MS MARCO: load_dataset("ms_marco", "v1.1", split="train[:15000]")
- Parse dataset structure:
 - Extract all passage texts from item["passages"]["passage_text"] — flatten into a single list
 - Build eval set: for each item where any(is_selected) is True, extract query + selected passage
 - Deduplicate passages using list(set(...)), cap corpus at 10,000 passages
 - Cap eval queries at 200
- Print: corpus size, eval query count, one sample query + its answer passage

Phase 2 — Corpus Encoding

- Load model: SentenceTransformer('all-MiniLM-L6-v2')
- Encode all 10,000 passages:

```
corpus_embeddings = model.encode(
    passages,
    batch_size=128,
    show_progress_bar=True,
    convert_to_numpy=True,
    normalize_embeddings=True  # L2 normalize → cosine sim = dot product
).astype('float32')
```

- Print encoding time and shape: (10000, 384)
- Understand and document: why normalize_embeddings=True lets you use IndexFlatIP instead of IndexFlatL2

Phase 3 — FAISS Index

- Build index using IndexFlatIP (inner product = cosine similarity after L2 normalization):

```
index = faiss.IndexFlatIP(d)  # d = 384
index.add(corpus_embeddings)
# Verify: index.ntotal == 10000
```

- Save index and passages to disk:

```
faiss.write_index(index, 'index/corpus_index.faiss')
np.save('index/passages.npy', np.array(passages))
```

- Reload index from disk and verify it still returns correct results — this is the production pattern

Phase 4 — Retrieval Function

- Implement a clean, reusable retrieval function:

```
def retrieve_top_k(query: str, k: int = 10) -> list:
    query_emb = model.encode([query], normalize_embeddings=True).astype('float32')
    scores, indices = index.search(query_emb, k)
    return [(passages[idx], float(scores[0][i]))
            for i, idx in enumerate(indices[0])]
```

- Test on 3 manually written queries (not from the dataset) — inspect results for sanity
- Print top-3 results per test query with similarity scores

Phase 5 — Evaluation: MRR & Recall@k

This is the most important section — it proves the system actually works and gives you the standard IR metrics that every search/RAG interview question will reference.

Metric Definitions

Metric	What it measures	Formula
MRR@k	How early the correct answer appears in top-k results	Mean of 1/rank (0 if not found)
Recall@k	Did the correct answer appear anywhere in top-k?	Binary: 1.0 or 0.0 per query

MRR intuition: answer at rank 1 → score 1.0. Rank 2 → 0.5. Rank 5 → 0.2. Not found → 0.0.

- Run evaluate_retrieval() on all 200 eval queries for k = 1, 5, 10
- Print results table for all k values
- Save bar chart of MRR@k and Recall@k to reports/figures/retrieval_metrics.png

Phase 6 — Keyword Baseline Comparison

- ```
def keyword_baseline(query: str, k: int = 10) -> list:
 query_words = set(query.lower().split())
 scored = []
 for i, passage in enumerate(passages):
 passage_words = set(passage.lower().split())
 overlap = len(query_words & passage_words)
 scored.append((i, overlap))
 scored.sort(key=lambda x: -x[1])
 return [(passages[idx], float(score)) for idx, score in scored[:k]]
```

- | Metric      | Keyword Baseline | SBERT + FAISS |
|-------------|------------------|---------------|
| MRR@1       | fill in          | fill in       |
| MRR@5       | fill in          | fill in       |
| MRR@10      | fill in          | fill in       |
| Recall@5    | fill in          | fill in       |
| Recall@10   | fill in          | fill in       |
| Avg Latency | fill in ms       | fill in ms    |

## Phase 7 — Latency Benchmarking

## Phase 8 — W&B; Logging

## Phase 9 — README

- ```
Query --> SentenceTransformer (all-MiniLM-L6-v2) --> 384-dim vector
      |
Corpus (10K passages) --> pre-encoded embeddings --> FAISS IndexFlatIP
      |
      Top-k passages + scores
```

- Full comparison table: Keyword baseline vs SBERT (MRR@k + Recall@k with real numbers)
- Latency numbers: avg, P50, P95, P99
- How to rebuild the index: `pip install -r requirements.txt` -> run notebook cells 1-4
- Key finding: 2-3 sentences on what the metrics tell you — where does SBERT win most?
- Production scaling note: what changes at 1M+ passages (IVFFlat, quantization, Pinecone)
- W&B; run link (make public)

■ Definition of Done

- MS MARCO loads correctly — corpus of 10,000+ unique passages confirmed
- 200 eval queries with known answer passages extracted
- all-MiniLM-L6-v2 encodes all passages — shape (10000, 384) confirmed
- `normalize_embeddings=True` used — reason documented in notebook
- FAISS IndexFlatIP built with all 10K vectors — `index.ntotal == 10000` verified
- Index saved to `index/corpus_index.faiss` and reloaded successfully
- `retrieve_top_k()` function implemented and tested on 3 custom queries
- MRR@1, MRR@5, MRR@10 computed and printed for SBERT
- Recall@5, Recall@10 computed and printed for SBERT
- Keyword baseline built and evaluated with same MRR/Recall metrics
- Full comparison table populated with real numbers
- Latency benchmarked: avg, P50, P95, P99 (ms) printed
- Bar chart saved to `reports/figures/retrieval_metrics.png`
- W&B; run logged with all metrics
- README complete with architecture diagram + comparison table + latency + W&B; link
- `*.faiss` and `*.npy` added to `.gitignore`
- Notebook pushed to GitHub with clean structure

■ What This Lets You Say In Interviews

- "I built a semantic search system over 10,000 passages using Sentence Transformers and FAISS, evaluated with MRR and Recall@k — this is the core retrieval component of any RAG pipeline."
- "I normalized embeddings before indexing so that inner product in FAISS equals cosine similarity — this lets me use IndexFlatIP which is faster than recomputing cosine similarity at query time."
- "Keyword search found the correct answer in the top-10 only X% of the time. SBERT raised that to Y% — on queries with paraphrased answers that share no keywords with the passage, SBERT is the only thing that works."
- "I measured retrieval latency end-to-end — the FAISS search itself takes ~1ms. The bottleneck is query encoding, which takes ~20-30ms on CPU. In production you'd use a GPU or pre-compute query caches for common searches."
- "MRR rewards finding the answer early in the ranking — an answer at rank 1 scores 1.0 but at rank 5 scores only 0.2. Recall@k only asks whether the answer appeared at all. For a QA system I care more about MRR; for a candidate generation stage I care more about Recall@k."
- "To scale this to 1M+ passages, I'd switch from IndexFlatIP (exact brute-force) to IndexIVFFlat (approximate, clusters vectors into cells) — or use a managed service like Pinecone or Weaviate."